

Hackathon 2026 – Organized as part of SEMICON India

APPLIED MATERIALS Problem Statement

Drift-Sense

AI-Powered Navigation-Error Recovery for Wafer Inspection Tools

1. Background

Semiconductor wafers contain many repeated dies and highly repetitive microscopic structures. Inspection tools often need to revisit the same relative location on another die. Small errors caused by stage drift, vibration or thermal effects can make the tool land away from the intended site.

The recovery task is visual: use a previously captured 100x close-up reference image to locate the same structure in a wider 10x search image and return its centre coordinates.

SIMPLE EXAMPLE

Think of a close-up photo of one tile in a large patterned floor. Find that tile in a wider, noisier photo. If several tiles match, choose the valid match closest to the centre of the wider photo.

2. Problem Description

Participants are required to develop an AI-based navigation recovery solution capable of locating a high-resolution reference pattern inside a larger low-resolution search image.

The challenge consists of two images:

Reference Image (100x Magnification)

The reference image represents:

- The exact target location
- High-magnification view
- High-resolution capture
- Area previously inspected

Search Image (10x Magnification)

The search image represents:

- A wider field of view
- Lower magnification

- Larger physical area
- Contains the target location somewhere inside it

The same pattern visible in the reference image appears inside the search image at approximately 10× reduced scale.

The objective is to find the precise location of the reference pattern within the search image and return the center coordinates of the matching region.

Build a reproducible Python solution with two connected parts:

1. Generate realistic synthetic grayscale semiconductor image pairs representing either DRAM-style or FinFET-style structures.
2. Develop an algorithm that locates the 100x reference pattern inside the 10x search image and outputs the selected centre as (x, y).

The solution must handle the nominal 10:1 magnification difference, realistic SEM-style degradation, small scale/rotation changes and repeated patterns. Classical computer vision, machine learning, deep learning or a hybrid approach may be used.

3. Objective

- Generate realistic, literature-supported synthetic inspection data.
- Recover the intended navigation location accurately and automatically.
- Remain robust as noise, blur, scale variation and pattern ambiguity increase.
- Provide runnable code, measurable results, failure analysis and a clear solution presentation.

4. Scope & Key Requirements

A. Input and output

Item	Requirement
Reference image	1000 x 1000 grayscale image representing a 100x close-up view.
Search image	1000 x 1000 grayscale image representing a wider 10x view containing the target.
Scale relationship	Nominal 10:1. The sessions indicated robustness testing may include approximately 9:1 to 11:1.
Rotation	Small variations of about 1-2 degrees may occur.
Output	Predicted target-centre coordinates (x, y) in search-image pixels.
Coordinates	Origin (0, 0) is top-left; x increases right and y increases downward.



Item	Requirement
Multiple matches	If several valid matches exist, select the one whose centre is closest to the search-image centre.

B. Synthetic dataset creation

- Choose either DRAM-style or FinFET-style structures; both are judged equally.
- Use only public structural knowledge and participant-created synthetic data. Do not use confidential fab data.
- Generate the paired reference image, search image, true target-centre coordinates and per-pair metadata.
- Store the random seed, architecture, transformations, noise settings, scale, rotation and ground truth for every pair.
- Justify structures, noise and augmentations using at least 2-3 credible public sources; include citations in the PPT and documentation.

Possible degradations discussed in the sessions include blur/astigmatism, shot noise, drift or jitter, contrast/gamma change, charging streaks, salt-and-pepper noise, spot-size blur and feature loss during downsampling. No single noise model is mandatory; choices must be technically justified.

C. Localization solution

- Implement the evaluation solution in Python.
- Account explicitly for the scale difference instead of relying on an accidental match.
- Process a pair or evaluator-provided batch without manual source-code changes.
- Return the selected centre coordinates and provide a repeatable score or confidence where possible.
- Measure computation time and explain at least one genuine failure case.
- Deep learning is not mandatory. Pretrained models are allowed if all weights and dependencies are disclosed and available.
- A web application is not required. A single notebook is not sufficient as the only runnable submission.

D. Validation requirements

Teams should validate on at least 30 varied, independently generated pairs and report:

- Euclidean localization error: $\sqrt{(x_{\text{pred}} - x_{\text{true}})^2 + (y_{\text{pred}} - y_{\text{true}})^2}$.
- Pass rate at 5-, 4-, 2- and 1-pixel thresholds, plus sub-pixel performance where supported.
- Mean, median and worst-case error.
- Runtime per image pair, with hardware, Python version and timing method.
- Results across multiple noise levels, target positions, scales and rotations.
- At least one visualized failure case with root-cause explanation.

IMPORTANT



The final evaluation utility, exact sub-pixel cutoff, test dataset instructions and runtime environment will take precedence when released.

5. Phase-wise Deliverables

Submit the following for the current solution-submission phase. Later test-data or finale instructions will be issued separately by the organizers.

Deliverable	What it must contain
1. Solution PPT/PPTX (mandatory)	Problem understanding, approach, architecture choice, synthetic-data method, degradations, localization method, experiments, threshold-wise results, runtime, failure analysis, citations, limitations and next steps.
2. Source code	Separate, documented Python code for dataset generation and localization/inference.
3. README	Environment setup, folder structure, exact commands, input/output examples, coordinate convention and assumptions.
4. Dependencies	requirements.txt or pip-freeze equivalent; include pretrained/model weights if used.
5. Results	Metrics, plots/overlays, runtime, robustness analysis and at least one failure case.
6. CSV/manifest	Reference path, search-image path, ground-truth x/y for generated cases, predicted x/y and per-pair generation metadata.
7. References	Public sources supporting semiconductor structures, image formation, noise and transformations.

Recommended submission GitHub folder

```
submission/  
  solution_presentation.pptx  
  README.md  
  requirements.txt  
  generate_dataset.py  
  localize.py  
  configs/  
  src/  
  model/           (if used)  
  results/  
  references/
```



6. Evaluation Parameters

The sponsor presentation states the following provisional framework:

Parameter	Weight	What evaluators will examine
Localization / inference	50%	Coordinate accuracy on sponsor test data and computation time.
Synthetic augmentation code	30%	Realism, diversity, reproducibility and literature-based justification.
Failure analysis / explainability	10%	Understanding of failure causes, especially repeated-pattern ambiguity.
RGB optical-image extension	Bonus	Optional generalization after completing the grayscale SEM task.
Remaining core weight	10% pending	Not defined in the supplied presentation; await the detailed rubric.

7. Recommended Solution PPT Structure

Idea submission template - https://i4c.in/wp-content/uploads/2026/07/Idea-Submission-Template_Hackathon-2026-1.pptx

1. Slide 1: Title, team and one-line solution summary.
2. Slide 2: Problem understanding and navigation-error context.
3. Slide 3: Proposed end-to-end workflow.
4. Slide 4: DRAM/FinFET synthetic-data design.
5. Slide 5: Noise, blur, scale and rotation modelling with citations.
6. Slide 6: Localization method and decision rule.
7. Slide 7: Implementation and execution commands.
8. Slide 8: Experiment setup and test diversity.
9. Slide 9: Threshold-wise accuracy, error statistics and runtime.
10. Slide 10: Robustness comparison and baseline/ablation.
11. Slide 11: Failure case, limitations and learnings.
12. Slide 12: Conclusion, next steps and repository/submission reference.

8. Student Workflow

1. Study public DRAM/FinFET structures and SEM imaging; save citations.
2. Create a parameterized image-pair generator with exact ground truth.
3. Build a simple scale-aware baseline before adding complexity.



4. Test normal and difficult cases: higher noise, repeated patterns, edge positions, 9:1-11:1 scale and 1-2° rotation.
5. Run at least 30 documented cases and calculate threshold-wise accuracy, error statistics and runtime.
6. Analyze failures, improve the method and state remaining limitations honestly.
7. Package the code and run the README commands in a clean environment before submission.
8. Confirm that all PPT claims match the submitted CSV, plots and code outputs.

9. Final Submission Checklist

- ☐ Mandatory solution PPT/PPTX included.
- ☐ 1000 x 1000 reference and search images used.
- ☐ 10:1 relationship implemented and documented.
- ☐ Top-left coordinate convention and closest-to-centre rule implemented.
- ☐ Python generator and Python localization code are separate and runnable.
- ☐ README contains exact setup and execution commands.
- ☐ Dependencies and all required weights are included.
- ☐ At least 30 varied cases are evaluated.
- ☐ 5-, 4-, 2- and 1-pixel pass rates are reported.
- ☐ Runtime, hardware and timing method are stated.
- ☐ At least one failure case is shown and explained.
- ☐ CSV/manifest contains paths, true coordinates, predictions and metadata.
- ☐ At least 2-3 credible public sources are cited.
- ☐ No proprietary data or hard-coded local paths are present.
- ☐ Submission was dry-run in a clean environment.

10. Applied Materials Session Notes

Problem Statement Webinar – 31 July 2026

- The core problem is cross-magnification localization in repetitive semiconductor patterns.
- Both input images are 1000 x 1000 pixels but represent different physical fields of view.
- Teams may choose DRAM or FinFET and must create synthetic data using public knowledge.
- Accuracy, synthetic augmentation, runtime and explainable failure analysis are central evaluation themes.

Key Concepts & Q&A – 6 August 2026

- Coordinate origin is top-left; output is the target centre in search-image pixels.
- Slight scale variation, 1-2° rotation and a noisier search image may be tested.



- Report performance at 5, 4, 2 and 1 pixels and sub-pixel level where supported.
- Deep learning and a web application are not mandatory; pretrained models are allowed.
- A CSV/manifest and per-pair metadata are expected for reproducibility.

11. Resource and Important Notes

Applied Materials starter resource: [Drift-Sense Synthetic Data – Hugging Face](#)

Problem Statement Explanation - https://www.youtube.com/watch?v=l_mYBGeoiXA

Q&A session - <https://www.youtube.com/watch?v=AA5Wb9FUACc&feature=youtu.be>

